

SIMATS ENGINEERING

Department of Computer Science and Engineering

Student Name: G.Lakshmi Mounika Course Code:	CSA15	Registration No.: 192372110 Assessment:	CO5 AT1
Course Name:	Cloud Computing and Big Data Analytics	Type:	Constraint-Based Problem
CO Statement:	Apply big data storage, integration, Hadoop, HDFS, and MapReduce concepts for distributed data processing		

Part A: HDFS Fundamentals – Lesson Notes

What is HDFS?

The Hadoop Distributed File System (HDFS) is a distributed storage system designed to reliably store very large files (hundreds of MB or more) across multiple machines, to stream those files at high bandwidth, and to tolerate node failures gracefully. Key design principles:

1. Fault tolerance through replication – files are replicated across multiple nodes.
2. Write-once-read-many access – simplifies data consistency.
3. High throughput over latency – optimized for batch processing, not real-time.

Lesson Note: HDFS vs. Traditional File Systems

HDFS is NOT a general-purpose file system like ext4 or NTFS. It trades random access and POSIX compliance for high throughput on large files and fault tolerance. A single HDFS file might be 100GB+, split into blocks across dozens of DataNodes.

HDFS Architecture

An HDFS cluster consists of three main components:

Component	Role	Count per Cluster
NameNode	Manages file system namespace; maintains file system tree + metadata for files/folders. Does NOT store actual data.	1 (or 1 primary + 1 secondary for HA)
Secondary NameNode	NOT a backup NameNode. Performs housekeeping (merges namespace image + edit logs). Prevents NameNode's edit logs from becoming too large.	1
DataNodes	Store the actual data blocks. Perform block creation, deletion, replication on instruction from NameNode.	Many (typically 10–100s)

Critical Insight: Single Point of Failure

The NameNode holds the entire file system namespace in RAM. If it crashes, the cluster is unusable until restart. This is why a Secondary NameNode (or HA pair) is essential in production.

Blocks & Replication

HDFS breaks each file into blocks (default 128MB or 256MB). Each block is replicated across multiple DataNodes. The replication factor (RF) controls how many copies exist.

Replication Factor	Fault Tolerance	Storage Overhead	Typical Use Case
1	None – single node failure loses data	100%	Non-critical data, dev/test
2	Survive 1 rack failure (if nodes on same rack); single node failure OK	200%	Development clusters
3	Survive 1 rack failure + 1 node in another rack	300%	Production (Hadoop default)

Rack Awareness Strategy

With RF=3: place 2 replicas on same rack (fast write); 1 replica on different rack (survival of rack-level disaster). This balances write speed (same rack) vs. fault tolerance (different racks).

Block Sizing & Data Locality

Block size (default 128MB) affects:

- Larger blocks → fewer blocks → fewer map tasks. May reduce parallelism.
- Smaller blocks → more blocks → more map tasks → better parallelism but more NameNode pressure.

Data locality principle: MapReduce scheduler tries to place mappers on nodes holding the input block. Three tiers:

1. Node-local (best): mapper runs on same node as block.
2. Rack-local (ok): mapper on same rack as block, different node.
3. Off-rack (worst): mapper elsewhere, data moves over network.

Part B: Main Scenario – Web Log Analysis Cluster

Problem Statement

Scenario: A company operates 100 web servers generating web access logs. Each server generates ~5GB of logs per day. The company needs to:

- Store 2 years of logs (36.5TB raw data)
- Run daily MapReduce jobs to analyze visitor patterns
- Survive single rack failure without data loss
- Minimize hardware costs while ensuring high availability

Lesson Note: Constraint-Based Design

Real-world systems have conflicting constraints: high availability wants more replication (higher cost); cost minimization wants less replication. Your job is to balance them under business constraints.

Step 1: Calculate Storage Requirements

Instructions: Fill in the worksheet below.

Metric	Value	Your Answer
Raw data (100 servers × 5GB/day × 365 × 2 years)	36,500 GB = 36.5 TB	_____ 36.5 TB
Replication factor assumed	3× (survive 1 rack fail)	_____ 3×
Total raw disk needed (raw data × RF)		_____ 109.5 TB
Overhead for metadata (~5% of raw)		_____ 5.5 TB
Usable capacity after overhead		_____ 115 TB

Guided Calculation

Total raw disk = 36.5 TB × 3 = 109.5 TB

Metadata overhead ≈ 109.5 × 0.05 = 5.5 TB

Usable capacity ≈ 109.5 + 5.5 = 115 TB

Step 2: Size the DataNode Hardware

Typical DataNode specs: 12TB disk, 64GB RAM. Design the cluster:

Item	Calculation	Your Answer
Number of DataNodes needed	115 TB ÷ 12 TB/node = ~10 nodes	_____ 10 DataNodes
Total RAM across cluster	10 nodes × 64 GB = 640 GB	_____ 640 GB
NameNode heap size (rule: 1GB per 1M files)	If ~50M files → 50GB NameNode heap	_____ 50 GB

Step 3: Replication & Rack Topology

Design the rack layout. Assume 2 racks (each can hold ~5 nodes).

Topology Layout

Rack 1: 5 DataNodes (DN1–DN5) Rack 2: 5 DataNodes (DN6–DN10) NameNode + Secondary NameNode: Separate machine (NOT on a DataNode).

Fill in the replication strategy:

Replica #	Placement	Rationale
1st replica	<u> </u> Rack 1	Why? Fast local access
2nd replica	<u> </u> Rack 1	Why? Improves local availability
3rd replica	<u> </u> Rack 2	Why? Protects against rack failure

Step 4: Fault Tolerance Analysis

Scenario: Rack 1 fails (all 5 DataNodes go down).

Instructions: Analyze data loss for different replica placements.

Expected Insight

If all 3 replicas are in Rack 1, losing Rack 1 means total data loss. The standard strategy spreads replicas: 2 in one rack, 1 in another, preventing rack-level loss.

Your Turn:

1. If we place replicas as: 2 in Rack 1, 1 in Rack 2, what happens when Rack 1 fails?

With 2 replicas in Rack 1 and 1 replica in Rack 2, a Rack 1 failure makes the two Rack-1 copies unavailable. The third replica in Rack 2 remains available, so there is no permanent data loss. HDFS can serve the data from Rack 2 and re-replicate the affected blocks to healthy DataNodes. Redundancy is temporarily reduced until recovery is complete.

Step 5: MapReduce Job Scheduling

A daily MapReduce job processes the logs: input = 5GB/day × 2 years = 36.5TB.

Metric	Calculation	Your Answer
Block size	128 MB (default)	<u> </u> 128 MB
Number of blocks	36.5 TB ÷ 128 MB = ~292,000 blocks	<u> </u> ~292,000
Number of mappers (≈ 1 per block)	~292,000 mappers	<u> </u> ~292,000
Data locality %	If 3 replicas well-distributed, expect 90%+ node-local	<u> </u> 90%+

Step 6: Cost-Benefit Analysis

Calculate total cost. Assume: \$500 per DataNode (hardware + network), \$1000 annual per DataNode (power, cooling, ops).

Cost Item	Calculation	Total
Initial hardware (10 DataNodes)	10 × \$500	<u> </u> \$5,000
Annual operations (10 DataNodes)	10 × \$1000	<u> </u> \$10,000
2-year total cost	(Hardware) + (Ops × 2)	<u> </u> \$25,000
Cost per TB stored (2 years)	(Total cost) ÷ (36.5 TB stored)	<u> </u> \$684.93/TB

Trade-off Question:

If you reduce RF from 3 to 2, you halve storage costs but lose rack-fault tolerance. Is this acceptable? Justify.

No. RF=3 should be retained because the requirement is to survive a complete rack failure without data loss. RF=2 lowers storage cost but may lose all copies if both replicas are placed in the failed rack. The extra cost of RF=3 is justified by higher availability and data protection.

Part C: Mini-Problems

Mini-Problem 1: Scaling for Peak Traffic

In Month 6, the company adds 50 new web servers (150 total), doubling log generation to 10GB/server/day. How does the HDFS cluster scale?

- New annual raw data: $150 \text{ servers} \times 10\text{GB/day} \times 365 \text{ days} = 547.5 \text{ TB/year}$ (at 2-year scale: 1095 TB)

Your task: Calculate new DataNode count, storage costs, and replication strategy.

Two-year raw data = 1,095 TB. With RF=3, replicated storage = 3,285 TB. Adding 5% metadata gives about 3,449 TB. At 12 TB per DataNode, about 288 DataNodes are required. Hardware cost = \$144,000; annual operations = \$288,000; two-year total = \$720,000. Keep RF=3 and distribute replicas across racks to preserve rack-failure tolerance.

Mini-Problem 2: Block Size Tuning

Experiment: What if we use 256MB blocks instead of 128MB?

Implications:

- Fewer blocks → fewer map tasks → less parallelism
- Fewer blocks → less NameNode memory

Estimate: With 256MB blocks, how many mappers for the 36.5TB dataset? Pros and cons?

With 256 MB blocks, the estimated mapper count is about 146,000. Pros: fewer blocks, fewer map tasks, and lower NameNode metadata usage. Cons: less parallelism and less fine-grained scheduling than 128 MB blocks.

Mini-Problem 3: Failure Recovery Timeline

A DataNode in Rack 1 crashes (hardware failure). HDFS detects missing replicas and re-replicates the blocks to other DataNodes. Estimate the re-replication time:

- DataNode disk: 12 TB
- Network bandwidth (typical): 1 Gbps

How long does it take to re-replicate 12 TB from peer DataNodes?

12 TB = 96 Tb. At 1 Gbps, ideal transfer time = 96,000 seconds, about 26.7 hours. Actual re-replication may take longer because of disk speed, network overhead, protocol overhead, and other cluster traffic.

Mini-Problem 4: Compression Trade-offs

Compress the logs with Gzip (typical compression ratio 3:1 for text logs). How does this affect storage and MapReduce job performance?

New compressed raw data: $36.5 \text{ TB} \div 3 = 12.17 \text{ TB}$

- Pro: Storage cost drops by 2/3
- Con: Mappers must decompress on-the-fly (higher CPU)

Decision: Should we compress? Justify based on the numbers.

Gzip reduces 36.5 TB to about 12.17 TB, saving about two-thirds of storage. It lowers storage cost but increases CPU work during decompression. Standard Gzip is not easily splittable, so MapReduce parallelism can fall. Compression is recommended because the storage saving is large, provided files are suitably chunked or a splittable format is used when parallelism is important.

Part D: Problem Bank – 4 Additional Scenarios

Scenario A: Real-Time Data Warehouse (E-commerce)

An e-commerce platform collects real-time user behavior (click streams, purchases) from mobile and web. Volume: 2TB/day. Need to query data within 1 hour for BI dashboards. Design an HDFS cluster with constraints: (1) RF=2 for cost, (2) 30-node cluster budget, (3) 1-day data retention (rolling 24-hour window). Deliverables: Cluster sizing, replication strategy, block size justification, 24-hour re-replication risk.

Cluster sizing: 1-day retention = 2 TB raw. With RF=2, required replicated storage is 4 TB before metadata. Use the 30-node budget for storage and query processing. Use 128 MB or 256 MB blocks; 256 MB lowers block and metadata counts, while 128 MB provides more parallelism. Place the two replicas on different racks. A node/rack failure close to the 24-hour expiry can temporarily reduce redundancy before old data is removed.

Scenario B: Backup & Archive (Regulatory Compliance)

A financial services firm must retain transaction logs for 7 years (compliance). Volume: 500GB/day. Budget: minimize cost (data is infrequently accessed). Design an HDFS cluster with: (1) RF=1 for cost, (2) separate cold-storage tier for logs older than 1 year, (3) quarterly backups to off-site. Deliverables: Cost analysis (7-year total), replication strategy, backup plan, data recovery RTO/RPO.

Seven-year raw retention = $500 \text{ GB/day} \times 365 \times 7 = 1,277.5 \text{ TB}$ (about 1.28 PB). Use RF=1 for the cost-minimized HDFS tier and move logs older than one year to cold storage. Perform quarterly off-site backups. The source scenario does not specify RTO/RPO targets, so exact values must be set by the business. Exact dollar cost also needs a hardware/storage price assumption.

Scenario C: IoT Sensor Network (Streaming + Batch)

IoT devices (temperature, humidity, pressure) stream data from 10,000 sensors at 1 reading/min (14.4M readings/day). Data volume: ~2GB/day (small payloads, high frequency). Need to: (1) retain 2 years, (2) detect anomalies in real-time (need fast access), (3) run weekly batch analytics. Deliverables: HDFS cluster design (NameNode heap for millions of small files!), block size & replica count, comparison with time-series DB (InfluxDB, etc.).

Two-year raw volume = $2 \text{ GB/day} \times 365 \times 2 = 1,460 \text{ GB}$ (about 1.46 TB). The major HDFS issue is the huge number of small readings/files, which can create severe NameNode metadata pressure. Aggregate readings into larger files before HDFS storage. Use 128–256 MB blocks and RF=3. A time-series DB such as InfluxDB is better for fast real-time queries, while HDFS is better for durable batch analytics; a hybrid design can support real-time anomaly detection and weekly HDFS analytics.

Scenario D: Multi-Tenant Cloud (SaaS)

A SaaS platform provides data analytics to multiple customers. Each customer's data is isolated in separate HDFS directories. Total: 100 customers, 1TB/customer on average. Constraint: (1) RF=3 for HA, (2) each customer must not access another's data. Deliverables: Cluster architecture (single cluster + multi-tenancy or separate clusters?), HDFS permissions & quotas, cost-per-customer estimate, cross-tenant security risks.

Use one HDFS cluster with multi-tenancy rather than 100 separate clusters. Keep each customer's data in a separate directory, apply HDFS permissions/ACLs, and enforce quotas. Total logical data = 100 TB; RF=3 gives about 300 TB replicated storage before metadata, or about 3 TB replicated per customer on average. Exact dollar cost per customer needs a hardware-price assumption. Security risks include incorrect permissions, broad service accounts, quota errors, and accidental cross-tenant access.

Evaluation Rubric

Criteria	Wt.	L4 (40–50)	L3 (30–39)	L2 (20–29)	L1 (0–19)
HDFS Concepts Understanding	20%	Accurate understanding of blocks, replication, NameNode role	Mostly correct; minor gaps	Basic understanding; some errors	Misinterprets core concepts
Constraint Analysis	20%	Identifies all constraints (cost, FT, scale); prioritizes well	Most constraints identified; minor prioritization gaps	Some constraints missed or poorly weighted	Ignores constraints or fails to analyze
Design & Calculations	30%	Correct cluster sizing, RF strategy, topology; justified	Mostly correct; minor arithmetic/logic gaps	Correct approach but calculation errors	Incorrect design or missing major elements
Trade-off & Risk Analysis	20%	Thorough analysis: cost vs. availability; risk mitigation	Good analysis; some trade-offs missed	Basic analysis; limited depth	Little or no trade-off analysis
Documentation & Clarity	10%	Clear diagrams, formulas, step-by-step reasoning	Mostly clear; minor formatting issues	Understandable but lacks detail	Unclear or poorly organized

Total Marks: 100 (20 + 20 + 30 + 20 + 10)